YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project – Low Level Design

On

Cloud Cost Intelligence Platform for Supply Chain

SUBMITTED BY

Muhammad Rijaz Schemnad - 23BCAICD067

BCA(AI, Cloud Computing, DevOps)

UNDER THE GUIDANCE OF

Industry Mentor: Shashank Korrram

Guided by: Sindhu

TABLE OF CONTENTS

1. Introduction.....02
2. Scope of the Document.....03
3. Intended Audience.....04
4. System Overview.....04
5. System Design.....05
   5.1 Application Design……..05
   5.2 Process Flow…….....06
   5.3 Information Flow……07
6. API Catalogue.....07
7. Data Models and Schemas.....07
8. Application Programming Interfaces……....08
9. Security Non-Functional Requirements.....09
10. References.....10

1. INTRODUCTION

The Cloud Cost Intelligence Platform for Supply Chain is a data-driven analytics system designed to process, clean, analyze, and visualize cloud billing data generated by supply chain operations. In modern enterprises, cloud services such as compute (EC2/VMs), storage (S3/Blob), database (RDS), networking (VPC), machine learning, analytics, serverless (Lambda), and containers (EKS/K8s) are heavily utilized across geographically distributed warehouses. Without a structured cost intelligence layer, organizations face uncontrolled cost escalation, resource waste, and inability to attribute cloud expenses to specific supply chain stages (warehouse, transport, delivery, order processing).

This project addresses these challenges by building an end-to-end pipeline using Python, Pandas, NumPy, Matplotlib, and Seaborn for data processing and analysis within Jupyter Notebooks, and a web-based dashboard (HTML/CSS/JavaScript with Chart.js) for interactive visualization. The platform ingests raw, unclean cloud billing spreadsheets, performs rigorous data cleaning and preprocessing, engineers meaningful features, conducts exploratory data analysis (EDA), builds a cost intelligence and alerting engine, and generates actionable recommendations.

Key Capabilities:

- Data cleaning and preprocessing of raw cloud billing records
- Cost driver analysis across services, regions, providers, and supply chain stages
- Inefficiency detection using statistical thresholds (IQR, standard deviation)
- Multi-cloud cost comparison (AWS, Azure, GCP, Oracle Cloud)
- Automated alerts and recommendations engine
- Interactive web-based dashboard with real-time filtering

2. Scope of the Document

- This Low-Level Design (LLD) document provides a detailed technical description of every module, function, algorithm, and data transformation within the Cloud Cost Intelligence Platform. It covers:
- Data Loading & Ingestion: How raw spreadsheet data is loaded, validated, and initially inspected.
- Data Preprocessing & Cleaning: Detailed cleaning logic including missing value handling, type conversions, text normalization, duplicate removal, and outlier detection using the IQR statistical method.
- Feature Engineering: Derivation of new analytical columns such as month, cost_per_unit, cloud_provider, supply_chain_stage, cost_per_order, and efficiency_score.
- Exploratory Data Analysis (EDA): Statistical and visual analysis of cost trends, regional distributions, service-wise breakdowns, provider comparisons, and correlation analysis.
- Cost Intelligence System: Automated threshold-based anomaly detection, cost spike identification, and environment waste analysis.
- Report Generation: Automated export of summary reports, alerts, and distribution files.
- Web Dashboard: Architecture of the interactive front-end visualization layer.

Out of Scope:

- Cloud API integration for live billing data retrieval
- User authentication and role-based access control
- Database-backed storage (uses file-based I/O)
- Deployment to cloud hosting environments

3. Intended Audience:

Audience	Purpose
Project Evaluators / Faculty	To assess the technical depth and implementation rigor of the final year project
Industry Mentors	To review the design against real-world FinOps and Cloud Cost Management practices
Developers / Future Contributors	To understand the codebase structure and extend or modify the platform
FinOps / Cloud Operations Teams	To understand the analytical logic and how recommendations are generated
Students / Peers	To use as a reference for similar data analytics or cloud cost management projects

4. System Overview:

The Cloud Cost Intelligence Platform operates as a batch-processing analytical pipeline with an offline web dashboard for visualization. The system is organized into three major layers:

4.1 Data Processing Layer (Jupyter Notebooks):

- cleaning.ipynb: The core notebook that handles the entire data pipeline from raw data ingestion through cleaning, feature engineering, EDA, cost intelligence, alerting, and report generation.
- Cost_Calculator_Dashboard.ipynb: A secondary notebook that loads the cleaned data, generates aggregated views, creates premium visualizations, exports Excel reports, and renders an in-notebook HTML dashboard.

4.2 Data Storage Layer (File System):

Input: Data/cloud_usage_unclean.xls — Raw, uncleaned cloud billing spreadsheet. Output: Results/

4.3 Presentation Layer (Web Platform):

- WebPlatform/index.html — Main HTML structure with sidebar filters and dashboard layout
- WebPlatform/css/style.css — Premium glassmorphism CSS design system
- WebPlatform/js/app.js — Client-side JavaScript for dynamic filtering and Chart.js visualizations
- WebPlatform/js/data.js — Offline dataset bundled as JavaScript for browser consumption

5. System Design

5.1 Application Design:

The application follows a sequential pipeline architecture where each processing stage depends on the output of the previous stage. This ensures data integrity and traceability across the entire workflow.

Module Decomposition:

Module	File	Responsibility
Data Ingestion	cleaning.ipynb (Section 1)	Load raw XLS, copy raw backup, inspect schema, standardize column names
Data Cleaning	cleaning.ipynb (Section 2)	Handle missing values, convert types, normalize text, remove duplicates, detect/remove outliers
Feature Engineering	cleaning.ipynb (Section 3)	Derive month, cost_per_unit, cloud_provider, supply_chain_stage
EDA & Visualization	cleaning.ipynb (Section 4)	Monthly trends, regional analysis, service breakdown, correlation matrix, provider analysis
Cost Intelligence	cleaning.ipynb (Section 5)	Statistical threshold alerting, cost spike detection, inefficiency scoring
Recommendations	cleaning.ipynb (Section 6)	Rule-based recommendation generation, final report assembly, artifact export
Dashboard Calculator	Cost_Calculator_Dashboard.ipynb	Aggregated views, pie charts, Excel export, in-notebook HTML dashboard
Web UI	WebPlatform/	Interactive filtering, metric computation, Chart.js visualizations

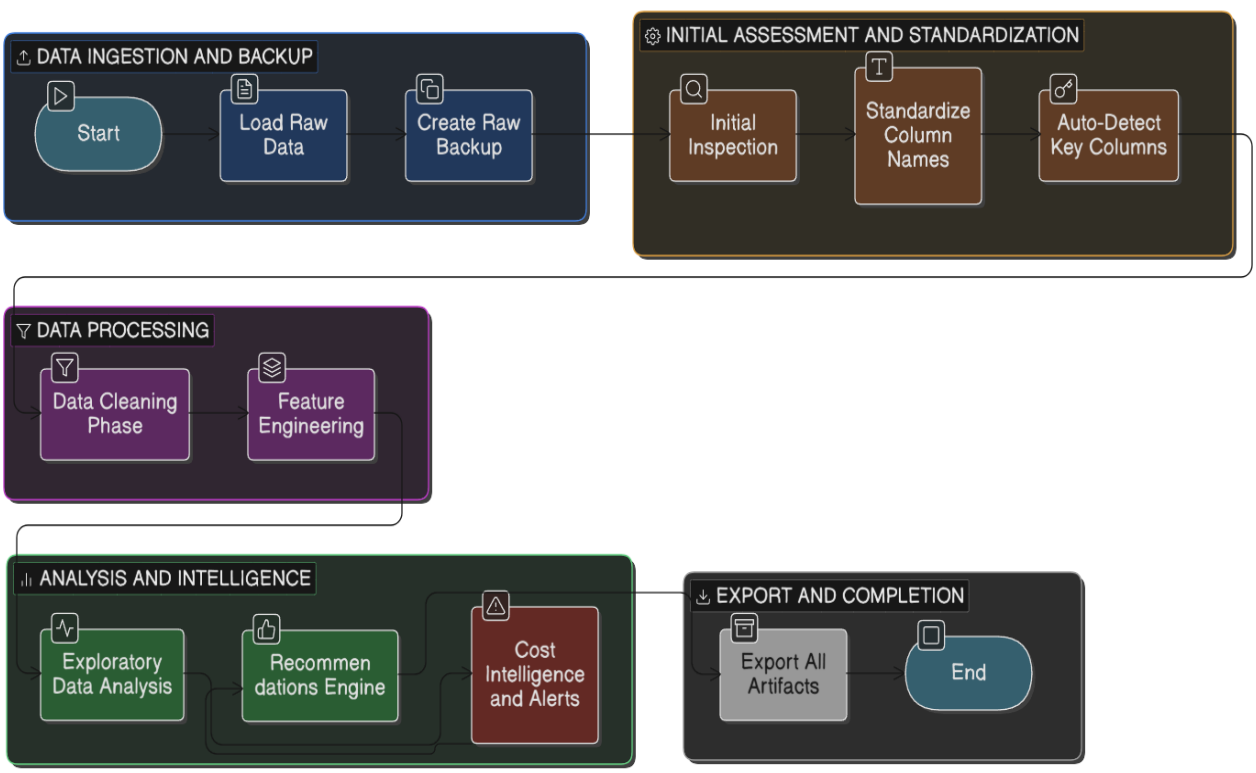
Technology Stack:

Component	Technology	Version
Programming Language	Python	3.10+
Data Manipulation	Pandas	Latest
Numerical Computing	NumPy	Latest
Visualization	Matplotlib, Seaborn	Latest
Development Environment	Jupyter Notebook	Latest
Web Frontend	HTML5, CSS3, JavaScript (ES6)	-
Charting Library	Chart.js	Latest (CDN)
Typography	Google Fonts (Inter)	-
Data Format	XLS/CSV (Spreadsheet)	-

5.2 Process Flow:

5.2.1 Overall Pipeline Process Flow:

The following flowchart illustrates the complete end-to-end pipeline of the Cloud Cost Intelligence Platform, from raw data ingestion to final report generation:



5.3 Information Flow:

The information flow describes how data is transformed and routed through the system at each stage. Data undergoes progressive refinement from raw billing records to actionable intelligence.

Information Flow Path:

Raw XLS File → pd.read_csv() → In-Memory DataFrame → Column Standardization → Auto-Detect & Rename → Missing Value Handling → Date/Numeric Type Conversion → Text Normalization → Duplicate Removal → IQR Outlier Detection → Feature Engineering → EDA Aggregations (groupby/agg) → Statistical Threshold Computation → Alert Generation → Recommendation Engine → File Export (XLS, TXT, CSV) → JavaScript Data Bundling → Web Dashboard Rendering

6. API Catalogue:

This project does not expose traditional REST APIs. Instead, it uses function-level interfaces within the Python notebooks and event-driven JavaScript functions in the web dashboard.

6.1 Python Function Interfaces (cleaning.ipynb):

Function	Input	Output	Description
assign_provider(row)	DataFrame row with region	String: AWS/Azure/GCP/Oracle Cloud	Assigns cloud provider based on region substring matching
map_stage(row)	DataFrame row with application, operation_type, shipment_volume, order_count	String: warehouse/transport/delivery/order_processing/other	Maps record to supply chain stage
Data Export Functions	Various DataFrames	.csv, .txt files in Results/	Export cleaned data, reports, alerts, recommendations

7. Data Design and Schemas

7.1 Raw Input Model

The exact dimensions of incoming cloud usage tables natively imported from the enterprise operations center.

Column Name	Expected Datatype	Description
Date	String/Object	Timestamps recorded in unpredictable regional formats
Project Name	String	Alphanumeric IDs corresponding to logic teams
Service Name	String	e.g., Relational Database Service, S3, Blob Storage
Resource Region	String	Datacenter geolocation hash
Cost	Float	Billed amount. Occasionally riddled with currency markers.

7.2 Cleaned Output Model:

The deterministic outputs that are locked in post-processing.

Column Name	Datatype	Domain Purpose
timestamp	Datetime64	Strict ISO compliant ISO-8601 formatting
project_team	String	Lowercased and standardized teams
total_billed	Float64	Uncorrupted float arrays
cloud_provider	Enforced Enum	AWS, GCP, AZURE or OTHER
supply_chain_stage	String	warehouse, transport, delivery, logic
efficiency_metric	Float64	Value representing compute efficacy (0 to 1 scale)

8. Application Programming Interfaces (APIs)

Due to the strict offline requirement of the system's operational parameters, traditional RESTful implementations (i.e. external POST/GET loops) are depreciated in favor of File-System interfaces and Javascript object bindings.

Virtual Endpoint	Method Type	Payload Description
fs://Data/cloud_usage_unclean.xls	Ingestion READ	Pulls raw organizational billing files from disk.
fs://Results/cleaned_data.csv	Pipeline WRITE	Persists the validated matrix onto the secure drive.
fs://WebPlatform/js/data.js	Javascript EXPORT	Translates Python structures into window.cloudData objects.
window.updateDashboard()	Frontend CALL	Fires chart rendering engine natively in browser runtime.

9. Security and Non-Functional Requirements

9.1 Performance Specs:

Data throughput must effectively handle datasets exceeding 150,000 continuous rows inside memory limits. Utilizing pandas vectorized arrays enables sub-20 second batch processing on typical corporate hardware structures. DOM rendering executes in under 200 milliseconds to avoid blocking the thread.

9.2 Security and Integrity Specs:

Because the system evaluates sensitive internal financial metrics regarding enterprise operations, data cannot leave the boundaries of the internal intranet execution shell.

Local Execution Only: By omitting cloud connectors directly within the ingestion layer, financial data is strictly confined to the disk it is executed on.

Defensive Data Parsing: `errors='coerce'` mechanisms actively prevent bad actors or broken files from executing injection-based or overflow-related crashes during conversion cycles.

9.3 Maintainability:

Architectural complexity was deliberately decoupled. Data Scientists can push changes to statistical models using solely Python routines within `cleaning.ipynb`. Independently, front-end developers can revamp the CSS structure and UI elements within `WebPlatform/` without modifying internal data generation dependencies.

12. References

#	Reference	Description
1	Pandas Documentation — pandas.pydata.org	Official Pandas library documentation
2	NumPy Documentation — numpy.org	NumPy library for numerical computations
3	Matplotlib Documentation — matplotlib.org	Matplotlib visualization library
4	Seaborn Documentation — seaborn.pydata.org	Seaborn statistical visualization
5	Chart.js Documentation — chartjs.org	Chart.js interactive charting library
6	IQR Method — Wikipedia	Statistical method for outlier detection
7	FinOps Foundation — finops.org	Industry framework for cloud financial management
8	AWS Well-Architected Framework	Cost Optimization Pillar best practices
9	Jupyter Notebook Documentation	Jupyter interactive development environment
10	Google Fonts — Inter	Inter typeface used in web dashboard